

CSE 401 – Numerical Methods

CT3 Revision Pack: Optimization, Regression & Interpolation

Compiled from NZR (Suchi) hand-notes

September 8, 2026

Roadmap of the syllabus (as taught, in order): Golden Section Search → Gradient Descent / Linear Regression → Newton's Method for Optimization → Foundations of Optimization (gradient, stationary points) → Constrained Optimization (Lagrangian & bordered Hessian) → Least-Squares Regression & Correlation → Multiple Linear Regression & its assumptions → Lagrange Interpolation → Linear Interpolation → (**v2 notes**) Newton's Divided-Difference Interpolation → Spline Interpolation (linear / quadratic / cubic).

CT syllabus (from sir's last page): the CT tests **Theory, Concept, and Math** – so know the *why* (concept), be able to *state* it (theory), and be able to *compute* it (math).

Contents

1	Golden Section Search	1
1.1	Setup and Assumptions	1
1.2	Algorithm (maximization)	1
1.3	Final estimate & stopping	2
1.4	Derivation of the Golden Ratio	2
2	Gradient Descent (GD) and Linear Regression	3
2.1	Motivation: what we optimize	3
2.2	Linear Regression using GD	3
2.3	GD of a single variable	3
2.4	GD for multiple variables	4
2.5	Problem of GD → Stochastic GD	4
3	Newton's Method for Optimization	4
3.1	From zero-finding to optimization	4
3.2	Limitations	4
3.3	Advantages (why not always GD?)	5
3.4	Derivation via Taylor's Theorem	5
3.5	Second-order (min) condition	5
3.6	Algorithm	5
3.7	Practical issues	6
4	Foundations of Optimization (Gradient & Stationary Points)	6
4.1	1-D examples – classifying stationary points	6
4.2	2-D example – the gradient vector	6

5	Constrained Optimization (Lagrange Multipliers)	6
5.1	The Lagrangian and its geometry	7
5.2	Parallel-gradient condition	7
5.3	Classifying the point: the bordered Hessian	7
5.4	Worked example	7
5.5	Advantages of the Lagrange-multiplier method	8
6	Linear Regression via Least Squares	8
6.1	Normal equations (closed-form)	8
6.2	Correlation Coefficient r	8
6.3	Computation table	9
7	Multiple Linear Regression	9
7.1	Cost function (Mean Squared Error)	9
7.2	Gradient Descent update	9
7.3	The five assumptions	9
8	Lagrange Interpolation	10
8.1	Statement	10
8.2	Proof	10
8.3	Compact form (Lagrange polynomial)	11
9	Linear Interpolation	11
9.1	Derivation	11
9.2	Choosing the bracketing points	11
10	Newton's Divided-Difference Interpolation (v2 notes)	11
10.1	The polynomial form	12
10.2	The coefficients are “divided differences”	12
10.3	The divided-difference table (this is what you build in the exam)	12
10.4	Proof of the recurrence (sketch)	13
11	Spline Interpolation (v2 notes)	13
11.1	Why we need splines – Runge's Phenomenon	13
11.2	Linear spline	13
11.3	Quadratic spline	13
11.4	Cubic spline (the important one)	14
	Quick Formula Sheet	14

1 Golden Section Search

Type: An **optimization technique** – an efficient way to locate the extremum (max/min) of a *unimodal* function. It is directly related to the **Golden Ratio**.

1.1 Setup and Assumptions

- Choose a lower bound x_L and an upper bound x_u that *bracket* (span) a single maximum.
- **Key assumption:** there is *exactly one* extremum in $[x_L, x_u]$ (the function is unimodal on the interval). **The method fails if there are multiple extrema.**

The Golden Ratio and step distance:

$$R = \frac{\sqrt{5} - 1}{2} \approx 0.618, \quad d = R(x_u - x_L).$$

1.2 Algorithm (maximization)

1. Evaluate the endpoints: $f_L = f(x_L)$, $f_u = f(x_u)$.
2. Compute two interior points using the golden distance d :

$$x_1 = x_u - d, \quad f_1 = f(x_1); \quad x_2 = x_L + d, \quad f_2 = f(x_2).$$

3. Locate the maximum's region:

$$f_1 > f_2 \Rightarrow \text{maximum lies to the left}, \quad f_1 < f_2 \Rightarrow \text{maximum lies to the right}.$$

4. Reassign points to shrink the interval and reuse one previously computed function value:

- If $f_1 < f_2$ (max to right): $x_L = x_1$, then $x_1 = x_2$, $f_1 = f_2$; x_u, f_u stay the same. New point needed: $x_2 = x_L + d$.
- If $f_1 > f_2$ (max to left): $x_u = x_2$, then $x_2 = x_1$, $f_2 = f_1$; x_L, f_L stay the same. New point needed: $x_1 = x_u - d$.

5. Recompute only the *one* missing interior point and its function value.

6. Repeat until convergence.

Efficiency (the whole point of "golden"): the first iteration costs **2 interior function evaluations** (4 total counting endpoints), but *every later iteration needs only 1 new function call* because the golden placement makes one new point land exactly on a previously evaluated point. This minimizes the number of (potentially expensive) function evaluations.

1.3 Final estimate & stopping

Once converged, report the optimum as the interval midpoint:

$$x_{\text{opt}} = \frac{x_u + x_L}{2}, \quad f_{\text{opt}} = f(x_{\text{opt}}).$$

Convergence (stopping) criterion:

$$\left| 2 \frac{x_u - x_L}{x_u + x_L} \right| < \varepsilon_{\text{tol}}.$$

1.4 Derivation of the Golden Ratio

Define the interval lengths in one iteration:

$$\ell_0 = x_u - x_L, \quad \ell_1 = x_2 - x_L, \quad \ell_2 = x_u - x_2.$$

We require that the points of the *next* iteration fall on top of points of the previous iteration (so we can reuse a value). This gives two conditions:

$$(i) \quad \ell_0 = \ell_1 + \ell_2, \quad (ii) \quad \frac{\ell_1}{\ell_0} = \frac{\ell_2}{\ell_1}.$$

Substitute (i) into (ii):

$$\frac{\ell_1}{\ell_1 + \ell_2} = \frac{\ell_2}{\ell_1} \implies \frac{\ell_1 + \ell_2}{\ell_1} = \frac{\ell_1}{\ell_2} \implies 1 + \frac{\ell_2}{\ell_1} = \frac{\ell_1}{\ell_2}.$$

Let $R = \ell_2/\ell_1$. Then $1 + R = 1/R$, i.e.

$$R^2 + R - 1 = 0 \implies R = \frac{-1 \pm \sqrt{1^2 - 4(1)(-1)}}{2(1)} = \frac{-1 \pm \sqrt{5}}{2}.$$

Taking the positive root: $R = \frac{\sqrt{5} - 1}{2} \approx 0.618$ – the Golden Ratio.

Exam Q – "Golden Ratio derivation / analysis": be ready to (a) state the unimodality assumption, (b) reproduce the two conditions $\ell_0 = \ell_1 + \ell_2$ and $\ell_1/\ell_0 = \ell_2/\ell_1$, and (c) solve the quadratic to get $R \approx 0.618$. Also explain *why* golden placement saves function evaluations (1 call per iteration after the first).

2 Gradient Descent (GD) and Linear Regression

2.1 Motivation: what we optimize

- **Linear regression:** optimize the *slope* and *intercept*.
- **Logistic regression:** optimize the "squiggle" (its slope/intercept).
- **t-SNE / clustering:** optimize the cluster arrangement.

Gradient Descent is a single, general tool that can optimize slope, intercept, clusters, squiggles, etc.

2.2 Linear Regression using GD

Model (treat slope as a constant for now while tuning the intercept):

$$\text{Height} = \underbrace{\text{Intercept}}_{\text{tune}} + \text{Slope} \times \text{Weight}.$$

1. Pick a random value for the intercept, e.g. Intercept = 0.
2. Evaluate fit with the **loss function** = Sum of Squared Residuals (SSR):

$$\text{SSR} = r_1^2 + r_2^2 + \cdots + r_n^2, \quad r_i = y_i - (\text{intercept} + \text{slope} \times x_i).$$

Plotting SSR vs. intercept gives a convex (bowl) curve; the bottom is the best intercept.

Why GD and not "just set derivative = 0"? For an optimum we need where slope = 0. Analytically setting the derivative to zero is fine for small problems, but often we *cannot* solve slope = 0 in closed form (many parameters / huge data). GD instead *takes steps* from an initial guess, doing only a few cheap calculations far from the optimum and more/finer ones as it approaches, until it reaches the best value.

2.3 GD of a single variable

$$\text{Step size} = \frac{d}{d(\text{intercept})}(\text{SSR}) \times \text{learning rate},$$

$$\text{New intercept} = \text{Old intercept} - \text{Step size}.$$

Schedule: the way the learning rate is changed from relatively large (early) to relatively small (later) is called a **schedule**.

Repeat until convergence, stop when either: (i) step size is close to zero, or (ii) a maximum number of steps is reached.

2.4 GD for multiple variables

When both slope s and intercept i are free (residual $y_i - (i + s x_i)$):

1. Take the derivative of the loss w.r.t. *each* parameter ($\partial/\partial i$ and $\partial/\partial s$).
2. Pick a random value for each parameter (e.g. $i = 0$, $s = 1$).
3. Plug the current parameter values into the derivatives.
4. Step size = (that derivative) $\times$ learning rate, for each parameter.
5. New param = old param – step size.
6. Repeat until convergence (step size ≈ 0 or max steps).

2.5 Problem of GD $\rightarrow$ Stochastic GD

Problem: what if there are millions of parameters/samples? Every step over all data is too expensive (data often has redundancy).

Stochastic Gradient Descent (SGD): randomly select a subset of the data at every step.

- **Strict definition:** use only *one* sample per step.
- **Common practice:** use a small subset (**mini-batch**) each step.
- **Advantages:** (i) best of both worlds; (ii) more stable estimates in fewer steps.
- **Cool thing:** when new data arrives we don't need to start from scratch – we can keep updating.

3 Newton's Method for Optimization

Root finding uses *1st-order* info; **optimization** uses *2nd-order* info (gradient and Hessian). Newton's method gives the *extremum* of a function.

3.1 From zero-finding to optimization

Zero-finding (root of f):

$$x_{t+1} = x_t - \frac{f(x_t)}{f'(x_t)}.$$

Optimization = find where the derivative is zero, so apply zero-finding to f' :

$$x_{t+1} = x_t - \frac{f'(x_t)}{f''(x_t)}.$$

Geometrically this makes a *2nd-order (quadratic) approximation* at x_t and jumps to that parabola's vertex.

3.2 Limitations

1. It finds *any* stationary point (derivative = 0): local minimum, local maximum, or saddle point – **not necessarily the global optimum**. A saddle/max has zero slope but is not a minimum.
2. It can move *toward a maximum instead of a minimum* if started in the wrong place (the update doesn't check the sign of f''). Remedy: use GD steps back and forth / check the curvature direction.

3.3 Advantages (why not always GD?)

1. Faster than GD (fewer iterations).
2. Not sensitive to the learning rate (no learning-rate tuning).
3. Number of correct digits roughly *doubles* each step (quadratic convergence $\Rightarrow$ very high accuracy fast).

Exam Q – "When do we use GD vs. Newton's method?"

Use **GD** when the problem size is large (typical ML problems – Hessian is huge/expensive).
Use **Newton's method** when the problem is small (Hessian is cheap to form and invert), to enjoy fast quadratic convergence.

3.4 Derivation via Taylor's Theorem

Smoothness requirement: let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be *sufficiently smooth* (continuous derivatives exist); otherwise the method fails.

Taylor expansion of f near a point a :

$$f(x) \approx f(a) + f'(a)(x - a) + \frac{1}{2}f''(a)(x - a)^2 + \frac{1}{3!}f'''(a)(x - a)^3 + \dots$$

We keep terms up to *2nd order*. For vectors (with gradient g and Hessian H):

$$f(x) = f(a) + g^\top(x - a) + \frac{1}{2}(x - a)^\top H(x - a), \quad g = \nabla f(a), \quad H = \nabla^2 f(a).$$

Expand the quadratic term (using $H^\top = H$ and $x^\top a = a^\top x$):

$$(x - a)^\top H(x - a) = x^\top Hx - 2a^\top Hx + a^\top Ha.$$

Write in standard quadratic form:

$$q(x) = \frac{1}{2}x^\top Hx + b^\top x + c, \quad b = g - Ha, \quad c = f(a) - g^\top a + \frac{1}{2}a^\top Ha.$$

Minimize by setting $\nabla q = 0$ (derivative zero):

$$Hx + b = 0 \Rightarrow x = -H^{-1}b = -H^{-1}(g - Ha) = a - H^{-1}g.$$

Here $g = \nabla f(a)$ (1st order) and $H = \nabla^2 f(a) = \left(\frac{\partial^2}{\partial x_i \partial x_j} f(a) \right)_{ij}$ (2nd order).

3.5 Second-order (min) condition

To be sure it is a *minimum*, $\nabla^2 q = H$ must be at least **positive semi-definite (PSD)** – every eigenvalue real and ≥ 0 . If every eigenvalue is > 0 it is **positive definite (PD)**.

$$\nabla^2 q = H, \quad \text{min if } H \text{ is PSD.}$$

If f is *convex*, H is PSD (bowl shape). A concave shape means the curvature points the "wrong way" – fix with a gradient-descent check.

3.6 Algorithm

1. Initialize $x_0 \in \mathbb{R}^n$.
2. Iterate: $x_{t+1} = x_t - H^{-1}g$, with $H = \nabla^2 f(x_t)$, $g = \nabla f(x_t)$.
3. Stop when the gradient is 0, i.e. $x_{t+1} = x_t$.

3.7 Practical issues

1. H may fail to be PD $\Rightarrow H^{-1}$ cannot be computed $\Rightarrow$ switch toward GD (**Levenberg–Marquardt algorithm** blends Newton and GD).
2. Rather than invert H directly, solve $Hy = g$ for y and use $x_{t+1} = x_t - y$ (cheaper / more stable).
3. Sometimes a **convergence-rate controller** $\alpha_t > 0$ is added:

$$x_{t+1} = x_t - \alpha_t H^{-1}g$$

to make convergence faster or slower (important knob).

4 Foundations of Optimization (Gradient & Stationary Points)

The **gradient** returns a vector field giving the *direction* and *magnitude* of the steepest slope for every possible input combination.

Basis of all optimization: (1) set up an objective function; (2) solve/search for locations of *zero gradient*.

4.1 1-D examples – classifying stationary points

To find a minimum: differentiate and set to 0; then check the second derivative.

Function	$f'(x) = 0$ gives	Nature
$J(x) = x^2$	$2x = 0 \Rightarrow x = 0$	minimum ($f'' = 2 > 0$)
$J(x) = -x^2$	$x = 0$	maximum
$J(x) = x^3$	$3x^2 = 0 \Rightarrow x = 0$	saddle point ($f'' = 6x$, sign depends on x)

The cubic is the cautionary case: derivative is 0 but it is neither a min nor a max – the second derivative $6x$ changes sign, so it is a saddle/inflection.

4.2 2-D example – the gradient vector

$$J(x) = x_1^2 + x_2^2, \quad \nabla J(x) = \begin{bmatrix} \partial J / \partial x_1 \\ \partial J / \partial x_2 \end{bmatrix} = \begin{bmatrix} 2x_1 \\ 2x_2 \end{bmatrix}.$$

Setting $\nabla J = 0$ gives the minimum at $(x_1, x_2) = (0, 0)$. Each component is the partial derivative w.r.t. that variable. For a function like $J(x) = x^4 - 6x^3 + 8x^2$, gradient descent follows the arrows "downhill" toward the minimum.

5 Constrained Optimization (Lagrange Multipliers)

Goal: find the min/max of an objective *subject to a constraint* – this is harder than the unconstrained case.

$$\text{Minimize (or Maximize) } J(x) \quad \text{subject to} \quad C(x) = 0.$$

5.1 The Lagrangian and its geometry

Form the **Lagrangian** by combining objective + $\lambda \times$ constraint, and set its gradient to zero:

$$\nabla (J(x) + \lambda C(x)) = 0 \Rightarrow \text{solution},$$

where λ is the **Lagrange multiplier**.

Geometric idea (contour plot – a 3-D surface drawn in 2-D): the constrained optimum is the point where the constraint path is *parallel to a contour* of J , equivalently *perpendicular to the gradient* of J .

5.2 Parallel-gradient condition

At the optimum the gradient of the objective and the gradient of the constraint point in the *same direction* (are parallel). Example:

$$J(x) = x_1^2 + x_2^2, \quad C(x) = x_1 + x_2 + 3, \quad \nabla C(x) = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Since $\nabla J \parallel \nabla C$, scale one to match the other:

$$\nabla J(x) = k \nabla C(x) \implies \nabla J(x) - k \nabla C(x) = 0 \implies \nabla(J(x) + \lambda C(x)) = 0.$$

That is exactly the stationarity of the Lagrangian.

5.3 Classifying the point: the bordered Hessian

The parallel-gradient condition only finds a *stationary* point (min / max / saddle). To classify it we use the **bordered Hessian** of $\pi = J(x) + \lambda C(x)$:

$$H(\pi) = \begin{bmatrix} \frac{\partial^2 \pi}{\partial \lambda^2} & \frac{\partial^2 \pi}{\partial \lambda \partial x} \\ \left(\frac{\partial^2 \pi}{\partial \lambda \partial x} \right)^\top & \frac{\partial^2 \pi}{\partial x^2} \end{bmatrix} = \begin{bmatrix} 0 & \frac{\partial C}{\partial x} \\ \left(\frac{\partial C}{\partial x} \right)^\top & \frac{\partial^2 \pi}{\partial x^2} \end{bmatrix}.$$

The top-left is 0 because $\frac{\partial}{\partial \lambda} \left(\frac{\partial}{\partial \lambda} (J + \lambda C) \right) = \frac{\partial}{\partial \lambda} C(x) = 0$; the border row/column are the constraint gradients $\partial C / \partial x_j$. In full n -variable form:

$$H = \begin{bmatrix} 0 & \partial C / \partial x_1 & \cdots & \partial C / \partial x_n \\ \partial C / \partial x_1 & \partial^2 \pi / \partial x_1^2 & \cdots & \partial^2 \pi / \partial x_1 \partial x_n \\ \vdots & \vdots & \ddots & \vdots \\ \partial C / \partial x_n & \partial^2 \pi / \partial x_1 \partial x_n & \cdots & \partial^2 \pi / \partial x_n^2 \end{bmatrix}.$$

5.4 Worked example

Note: if the constraint is given as $C(x) = g$, first rewrite it as $C(x) - g = 0$, then proceed. With $J = x_1^2 + x_2^2$, $C = x_1 + x_2 + 3$:

$$\pi = x_1^2 + x_2^2 + \lambda(x_1 + x_2 + 3).$$

Stationarity:

$$\frac{\partial \pi}{\partial x_1} = 2x_1 + \lambda = 0, \quad \frac{\partial \pi}{\partial x_2} = 2x_2 + \lambda = 0, \quad \frac{\partial \pi}{\partial \lambda} = x_1 + x_2 + 3 = 0.$$

Solving: $\boxed{x_1 = -1.5, \quad x_2 = -1.5, \quad \lambda = 3.}$

5.5 Advantages of the Lagrange-multiplier method

1. Converts a *constrained* problem into an *unconstrained* one.
2. λ carries useful *scale* information.
3. It shows how the objective changes when the constraint changes (λ = sensitivity / shadow price).

Exam Q – "Why the bordered Hessian?" To determine whether the stationary point found by the Lagrangian is a *minimum*, *maximum*, or *saddle point*. (Recall: the gradient only gives the steepest slope for every combination – it locates stationary points but doesn't classify them.)

6 Linear Regression via Least Squares

Fit $y = mx + b$ so that all data points are *as close as possible* to the line (small residuals).

$$\text{Error} = \text{real} - \text{predicted} = y_i - (mx_i + b).$$

Sum of Squared Errors (squared so that $+/-$ errors don't cancel):

$$\text{SSE} = \sum_{i=1}^n (y_i - mx_i - b)^2.$$

6.1 Normal equations (closed-form)

Set the partial derivatives to zero:

$$\begin{aligned} \frac{\partial \text{SSE}}{\partial b} = 0 &\implies b = \frac{\sum y - m \sum x}{n}, \\ \frac{\partial \text{SSE}}{\partial m} = 0 &\implies m = \frac{n \sum xy - \sum x \sum y}{n \sum x^2 - (\sum x)^2}. \end{aligned}$$

Exam tip from the notes: the full *proof* of these formulas may be asked in the exam – practice differentiating SSE w.r.t. b and m and solving the two normal equations.

6.2 Correlation Coefficient r

Describes the *strength and direction* of the (linear) relation between two variables x and y :

$$r = \frac{n \sum xy - \sum x \sum y}{\sqrt{[n \sum x^2 - (\sum x)^2] [n \sum y^2 - (\sum y)^2]}}.$$

- $r = +1$: perfect positive; $r = -1$: perfect negative line.
- $r = 0.8, 0.6$: fairly strong but points lie off the line; $r = 0$: no relation.
- Close to $\pm 1 \Rightarrow$ strong relation; close to 0 $\Rightarrow$ weak relation.
- r^2 (coefficient of determination): $r^2 = 1$ means the model explains 100% of the variance in the data; $r^2 = 0.6$ means 60% explained.

Correlation $\neq$ causation. A strong correlation does *not* mean one variable causes the other. Two things can move together by coincidence or because a hidden *third factor* influences both.

6.3 Computation table

Build a table of the running sums needed by the formulas:

$$\sum x, \quad \sum y, \quad \sum x^2, \quad \sum y^2, \quad \sum xy.$$

7 Multiple Linear Regression

Multiple features (e.g. height depends on weight, age, ...). Equation of a hyperplane in n dimensions:

$$y = w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n,$$

$$w_0 = \text{bias/intercept}, \quad w_1, \dots, w_n = \text{weights}, \quad x_1, \dots, x_n = \text{features}.$$

Dimensional intuition: 1D $\rightarrow$ point, 2D $\rightarrow$ line, 3D $\rightarrow$ plane, higher $\rightarrow$ hyperplane. (Without w_0 the fit is forced through the origin.)

Goal: find the weights that *minimize the error* between predicted and actual values.

7.1 Cost function (Mean Squared Error)

$$J(w_0, w_1, \dots, w_n) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2,$$

where m is the number of samples. Squaring stops $+/ -$ residuals from cancelling; the $\frac{1}{2m}$ factor removes scaling/shifting effects so the GD updates are clean.

7.2 Gradient Descent update

$$w_j \leftarrow w_j - \alpha \frac{\partial J}{\partial w_j}, \quad \alpha = \text{learning rate}.$$

Pick initial values for $w_0, \dots, w_n$, plug into the update, and iterate until convergence.

7.3 The five assumptions

1. **Linearity** – the relationship is linear in the parameters.
2. **Independence** – data points are independent of one another.
3. **Homoscedasticity** – the variance of the errors is constant (points don't fan out; no point sits extremely far).
4. **Normality** – the errors are normally distributed.
5. **No multicollinearity** – inputs are not linearly dependent on each other (features not redundant).

Exam Q – scenario-based (10–15 marks): "Is the assumption valid?"

- If the question says the *errors follow a Poisson distribution*, then **Normality** is violated $\Rightarrow$ multiple linear regression is **not** appropriate.
- If some points sit very far / the spread grows, **Homoscedasticity** is violated.
- If inputs are correlated with each other, **No multicollinearity** is violated.

Be able to name which assumption breaks in a given scenario and justify it.

8 Lagrange Interpolation

Regression finds a *best fit / explanation* and need not pass through every point (it tolerates noise/outliers). **Interpolation** instead builds a curve that *passes through all given points* exactly.

Degree rule: $(n + 1)$ points $\Rightarrow$ *at most* an n -th order polynomial ($\# \text{points} - 1$). It is "at most" because if the points happen to be collinear the interpolant collapses to a line. Trying to force a too-high-order polynomial through noisy data over-fits the noise/outliers. E.g. 3 points $\rightarrow$ quadratic; 4 points $\rightarrow$ cubic.

8.1 Statement

Given observations $x_0, x_1, \dots, x_n$ with corresponding values $y_0, y_1, \dots, y_n$ where $y = f(x)$:

$$\begin{aligned} f(x) &= \frac{(x - x_1)(x - x_2) \cdots (x - x_n)}{(x_0 - x_1)(x_0 - x_2) \cdots (x_0 - x_n)} y_0 \\ &+ \frac{(x - x_0)(x - x_2) \cdots (x - x_n)}{(x_1 - x_0)(x_1 - x_2) \cdots (x_1 - x_n)} y_1 + \cdots \\ &+ \frac{(x - x_0)(x - x_1) \cdots (x - x_{n-1})}{(x_n - x_0)(x_n - x_1) \cdots (x_n - x_{n-1})} y_n. \end{aligned}$$

8.2 Proof

Assume an n -th degree polynomial of the form (each term is an n -th order polynomial):

$$\begin{aligned} f(x) &= A_0(x - x_1)(x - x_2) \cdots (x - x_n) + A_1(x - x_0)(x - x_2) \cdots (x - x_n) \\ &+ \cdots + A_n(x - x_0)(x - x_1) \cdots (x - x_{n-1}). \end{aligned} \quad (1)$$

Substitute $x = x_0$ into (1). Every term except the first vanishes (each other term contains a factor $(x - x_0)$):

$$f(x_0) = A_0(x_0 - x_1)(x_0 - x_2) \cdots (x_0 - x_n),$$

and since $f(x_0) = y_0$,

$$A_0 = \frac{y_0}{(x_0 - x_1)(x_0 - x_2) \cdots (x_0 - x_n)}.$$

Similarly, substituting $x = x_1, \dots, x_n$ gives $A_1, \dots, A_n$. Putting all the A_k back into (1) yields $f(x)$.

8.3 Compact form (Lagrange polynomial)

$$f(x) = \sum_{j=1}^n \left[y_j \prod_{\substack{k=1 \\ k \neq j}}^n \frac{x - x_k}{x_j - x_k} \right].$$

(The index may start from 0 or 1 – it is only a naming convention.)

9 Linear Interpolation

Special case: fit a straight line $y = mx + b$ between the *two* data points that bracket the target x_m . Given a table:

x	x_1	x_2	x_3
y	y_1	y_2	y_3

find which two points surround x_m and draw the line through them.

9.1 Derivation

From two points (x_2, y_2) and (x_3, y_3) on the line:

$$y_2 = mx_2 + b \Rightarrow b = y_2 - mx_2, \quad y_3 = mx_3 + b.$$

Subtracting: $y_3 - y_2 = m(x_3 - x_2)$, so

$$m = \frac{y_3 - y_2}{x_3 - x_2}.$$

For the interpolated point (x_m, y_m) on the same line, $y_m = mx_m + b = mx_m + y_2 - mx_2$, hence $y_m - y_2 = m(x_m - x_2)$ and $m = \frac{y_m - y_2}{x_m - x_2}$. Equating the two slope expressions:

$$\boxed{\frac{y_3 - y_2}{x_3 - x_2} = \frac{y_m - y_2}{x_m - x_2}} \quad (\text{equation of linear interpolation}).$$

9.2 Choosing the bracketing points

Always use the two tabulated points that *surround* the query x :

- For an x_m lying between x_2 and x_3 , use $(x_2, y_2), (x_3, y_3)$.
- For an x_n lying between x_1 and x_2 , use $(x_1, y_1), (x_2, y_2)$:

$$\frac{y_2 - y_1}{x_2 - x_1} = \frac{y_n - y_1}{x_n - x_1}.$$

10 Newton's Divided-Difference Interpolation (v2 notes)

Another way to build the *same* interpolating polynomial that passes through all points – but organised so that **adding a new point does not force you to redo old work**. That reuse is its whole selling point over Lagrange.

10.1 The polynomial form

$$P_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots + a_n \prod_{i=0}^{n-1} (x - x_i).$$

Build it up point by point (each new point just adds *one* new term):

$$P_0(x) = a_0 \text{ (1 point } \rightarrow \text{ no } x), \quad P_1(x) = a_0 + a_1(x - x_0) \text{ (2 points } \rightarrow \text{ line)},$$

$$P_2(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) \text{ (3 points } \rightarrow \text{ quadratic)}.$$

10.2 The coefficients are “divided differences”

Plug $x = x_0, x_1, x_2, \dots$ one at a time; each collapses to give one coefficient:

$$a_0 = y_0, \quad a_1 = \frac{y_1 - y_0}{x_1 - x_0} = f[x_1, x_0] \text{ (1st divided difference),}$$

$$a_2 = \frac{f[x_2, x_1] - f[x_0, x_1]}{x_2 - x_0} = f[x_2, x_1, x_0] \text{ (2nd d.d.).}$$

Recurrence relation (the master formula – memorize this one):

$$f[x_0, x_1, \dots, x_{n+1}] = \frac{f[x_1, \dots, x_{n+1}] - f[x_0, \dots, x_n]}{x_{n+1} - x_0}.$$

Trick to remember: “(right group – left group) over (last x – first x).” Numerator = drop the first node vs. drop the last node; denominator = the two outermost x ’s.

Why Newton over Lagrange? – the recalculation-free property. The coefficients $a_0, a_1, \dots$ stay the *same* when a new data point (x_{n+1}, y_{n+1}) arrives; you only compute *one* extra term and add it. Lagrange would make you rebuild every basis polynomial from scratch.

10.3 The divided-difference table (this is what you build in the exam)

Lay the data in a triangular table; each entry = (the two diagonal neighbours to its left, subtracted) $\div$ (the x ’s spanning it):

$$\begin{array}{c|c|c|c} x_0 & y_0 & & \\ & f[x_0, x_1] & & \\ x_1 & y_1 & & \\ & f[x_1, x_2] & f[x_0, x_1, x_2] & \\ x_2 & y_2 & & \\ & f[x_2, x_3] & f[x_1, x_2, x_3] & f[x_0, x_1, x_2, x_3] \\ x_3 & y_3 & & \end{array}$$

The **top diagonal** entries are exactly $a_0, a_1, a_2, a_3, \dots$

- **Edge case / advantage:** to add a new point x_4, y_4 , just append a new bottom row and fill one new lower diagonal – old numbers are untouched (“ x_m without recalc, x_4, y_4 add”).
- **Equally-spaced nodes (equi-difference):** if x ’s are equally spaced with step h ($x_2 - x_1 = h$, $x_3 - x_1 = 2h$, $\dots$, e.g. census years 2000, 2001, $\dots$), you can switch to simpler *forward/backward difference* formulas.

10.4 Proof of the recurrence (sketch)

Let $P_n(x)$ interpolate $x_0 \rightarrow x_n$ (n -th order) and $L_n(x)$ interpolate $x_1 \rightarrow x_{n+1}$ (n -th order). Combine them into the $(n+1)$ -th order interpolant:

$$P_{n+1}(x) = \frac{(x - x_0)L_n(x) + (x_{n+1} - x)P_n(x)}{x_{n+1} - x_0}.$$

Check the endpoints and any interior node:

$$x = x_0 : P_{n+1}(x_0) = P_n(x_0) = y_0; \quad x = x_{n+1} : P_{n+1}(x_{n+1}) = L_n(x_{n+1}) = y_{n+1};$$

$$x = x_k \text{ (interior)} : P_{n+1}(x_k) = P_n(x_k) = y_k.$$

So P_{n+1} interpolates all $n+2$ points. Equating the coefficient of the highest power x^{n+1} on both sides yields exactly the divided-difference recurrence above. (Full algebra in the Maths & Proofs booklet.)

11 Spline Interpolation (v2 notes)

Global vs local. Newton & Lagrange are *global*: all points feed one big polynomial, and $(n+1)$ points force an n -th order polynomial. **Splines are local**: use a *subset* of data and join short low-order pieces – one small curve between each pair of neighbouring points.

11.1 Why we need splines – Runge’s Phenomenon

As you add data points, a single global polynomial’s **order shoots up**, and a high-order polynomial **oscillates wildly, worst at the edges** of equally spaced points. This blow-up is **Runge’s Phenomenon** (classic example: interpolating $\frac{1}{1+x^2}$ – the fit wiggles and can even go negative where the function never does). **Fix**: don’t fit one big curve; fit many small local pieces = splines.

11.2 Linear spline

Just connect consecutive points with straight lines:

$$P_1(x) = y_1 + \frac{y_2 - y_1}{x_2 - x_1}(x - x_1), \quad P_2(x) = y_2 + \frac{y_3 - y_2}{x_3 - x_2}(x - x_2), \dots$$

To estimate at a query, use the piece whose interval contains it. **Edge case / weakness**: *no curvature* – the joints are sharp corners (not smooth). That motivates higher-order splines.

11.3 Quadratic spline

Each piece is a parabola $P_i(x) = a_i x^2 + b_i x + c_i$. With n splines there are **$3n$ unknowns**, so we need $3n$ equations. **Memory hook**: **3 knobs per parabola** $\Rightarrow 3n$.

1. **Pass through known points** (each interior point is shared by two pieces): $P_1(x_1) = y_1, P_1(x_2) = y_2, P_2(x_2) = y_2, P_3(x_3) = y_3, \dots \Rightarrow 2n$ equations.
2. **Smoothness** – first derivatives match at interior points: $P'_1(x_2) = P'_2(x_2), \dots \Rightarrow (n-1)$ equations.
3. **Assumption / boundary** – second derivative zero at one end: $f''(P_1(x_1)) = 0 \Rightarrow 1$ equation.

Count check ($n=2$): $2n + (n-1) + 1 = 4 + 1 + 1 = 6 = 3n$. ✓

Smoothness meaning: at a joint both curves share the *same tangent* $\Rightarrow$ smooth; different tangents $\Rightarrow$ a kink (not smooth).

11.4 Cubic spline (the important one)

Each piece is a cubic $P_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i$ – **4 unknowns per piece** $\Rightarrow 4n$ unknowns. It **ensures continuity AND smoothness**. **Benefits**: (1) good function fit; (2) reduces oscillation; (3) smoothness and continuity.

1. **Pass through known points**: $2n$ equations.
2. **1st-derivative match** at interior points: $P'_i(x) = P'_{i+1}(x) \Rightarrow (n-1)$.
3. **2nd-derivative match** at interior points (maintains curvature continuity): $P''_i(x) = P''_{i+1}(x) \Rightarrow (n-1)$.

4. **Boundary assumptions** $\Rightarrow$ 2 equations, choose:

- **Natural cubic:** curvature zero at both ends, $P_1''(x_1) = 0$, $P_n''(x_{\text{end}}) = 0$.
- **Clamped cubic:** you are *given extra data* – the slope $\frac{dy}{dx}$ at the two ends.

Count check: $2n + (n - 1) + (n - 1) + 2 = 4n = \# \text{unknowns}$. $\checkmark$

The trade-off (one line to remember): higher spline order $\Rightarrow$ more smoothness, *but* more computation and more chance of error. Linear = simplest/kinks; cubic = smooth/costlier. Every spline needs its degrees of freedom pinned down by boundary assumptions.

Quick Formula Sheet

Topic	Key formula
Golden ratio / distance	$R = \frac{\sqrt{5}-1}{2} \approx 0.618$, $d = R(x_u - x_L)$
GSS interior points	$x_1 = x_u - d$, $x_2 = x_L + d$
GSS stopping	$\left 2 \frac{x_u - x_L}{x_u + x_L} \right < \varepsilon$
GD update	$w \leftarrow w - \alpha \partial J / \partial w$
Newton (opt.)	$x_{t+1} = x_t - H^{-1}g$ (1-D: $x_t - f'/f''$)
Min condition	H positive (semi)definite
Lagrangian	$\nabla(J(x) + \lambda C(x)) = 0$
LS slope	$m = \frac{n \sum xy - \sum x \sum y}{n \sum x^2 - (\sum x)^2}$
LS intercept	$b = \frac{\sum y - m \sum x}{n}$
Correlation	$r = \frac{n \sum xy - \sum x \sum y}{\sqrt{[n \sum x^2 - (\sum x)^2][n \sum y^2 - (\sum y)^2]}}$
MLR cost	$J = \frac{1}{2m} \sum (\hat{y}_i - y_i)^2$
Lagrange interp.	$f(x) = \sum_j y_j \prod_{k \neq j} \frac{x - x_k}{x_j - x_k}$
Linear interp.	$\frac{y_3 - y_2}{x_3 - x_2} = \frac{y_m - y_2}{x_m - x_2}$
Newton form	$P_n = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \dots$
Divided diff.	$f[x_0..x_{n+1}] = \frac{f[x_1..x_{n+1}] - f[x_0..x_n]}{x_{n+1} - x_0}$
Quadratic spline	$3n \text{ unknowns} = 2n + (n-1) + 1$
Cubic spline	$4n \text{ unknowns} = 2n + (n-1) + (n-1) + 2$
Natural cubic	$P'' = 0$ at both ends; Runge $\Rightarrow$ use splines

All the best for the exam!